

07-Redis

Redis 面试题

1. 数据结构

问：Redis 五种基本类型及底层实现？

答：

类型	底层实现	应用场景
String	SDS（简单动态字符串）	缓存、计数器、分布式锁
Hash	压缩列表 / 哈希表	对象缓存、购物车
List	压缩列表 / 双向链表	消息队列、时间线
Set	整数集合 / 哈希表	去重、共同关注
ZSet	压缩列表 / 跳表	排行榜

编码转换：元素少且值小时用压缩列表/listpack，超过阈值转哈希表/跳表。

问：Set 和 ZSet 的区别？ZSet 在项目里怎么用？

答：

对比	Set	ZSet
排序	无序	按 score 有序
底层	intset / hashtable	listpack / skiplist + dict
场景	去重、共同关注	排行榜、延迟队列

ZSet 项目用法： - 排行榜：score 为分数，ZREVRANGE 取 Top N - 延迟队列：score 为执行时间戳，ZRANGEBYSCORE 取到期任务 - 限流滑动窗口：score 为时间戳，按窗口清理

问：跳表是怎么实现的？为什么 Redis 用跳表不用红黑树？

答：跳表是多层有序链表，高层节点稀疏，查找时「跳跃」定位，复杂度 $O(\log N)$ 。

Redis 选择跳表： 1. 实现简单，范围查询友好（链表遍历） 2. 支持 $O(\log N)$ 查找和 $O(1)$ 头尾操作 3. 并发场景下红黑树旋转复杂

层高确定：随机数 < 0.25 则层数 +1，最大 64 层。

问：SDS 和 C 字符串的区别？

答：– O(1) 获取长度 (len 字段) – 杜绝缓冲区溢出 (自动扩容) – 减少内存重分配 (空间预分配、惰性释放) – 二进制安全 (可存 `\0`) – 兼容部分 C 字符串函数

2. 线程模型

问：Redis 为什么快？

答：1. 纯内存操作 2. 单线程执行命令，无锁竞争 (6.0 前) 3. IO 多路复用 (epoll) 处理并发连接 4. 高效数据结构 5. 避免上下文切换

问：Redis 为什么用 IO 多路复用？网络模型是什么？

答：单线程若阻塞等待每个客户端 IO，无法服务其他连接。IO 多路复用用一个线程监听多个 socket 就绪事件 (Linux `epoll`)。

流程：1. 客户端 fd 注册到 `epoll` 监听列表 2. `epoll` 等待任意 fd 可读/可写 3. 就绪后事件处理器读写 query buffer、执行命令

网络模型：– 6.0 前：单 Reactor 单线程 (命令执行 + 网络 IO 都在主线程) – 6.0 后：多 Reactor，网络 IO 多线程，命令执行仍单线程

BIO 后台线程：异步关闭文件、AOF `fsync`、lazy free 大 key 释放。

问：为什么使用 Redis？和 MySQL 比快在哪？

答：为什么用 Redis：– 纯内存，读写 μs 级 – 丰富数据结构 (String/Hash/ZSet 等) – 持久化、主从、集群 – 单线程无锁，实现简单

比 MySQL 快：

对比	Redis	MySQL
存储	内存	磁盘 + Buffer Pool
模型	KV + 结构	关系型 + SQL 解析
场景	缓存、计数、排行榜	持久化、复杂查询、事务

定位：Redis 是缓存/加速层，MySQL 是数据持久化主库，互补而非替代。

问：本地缓存和 Redis 缓存怎么选？

答：

对比	本地缓存 (Caffeine/Guava)	Redis
速度	最快 (进程内)	网络 RTT
一致性	多实例不一致	集中式一致
容量	受 JVM 堆限制	独立内存
场景	热 key、配置、字典	共享缓存、分布式锁

常见架构：本地缓存 (L1) + Redis (L2) + DB (L3)。

问：Redis 6.0 为什么引入多线程？

答：命令执行仍单线程，多线程仅用于 网络 IO（读请求、写响应），解决大并发下网络 IO 成为瓶颈的问题。默认关闭，需配置 `io-threads-do-reads yes`。

3. 持久化

问：RDB 和 AOF 的区别？

答：

对比	RDB	AOF
原理	定时快照	记录每条写命令
恢复速度	快	慢
数据安全	可能丢最后一次快照后的数据	更安全 (everysec)
文件大小	小 (压缩)	大
性能	fork 子进程有开销	写盘有开销

混合持久化 (Redis 4.0+)：AOF 重写时前半 RDB 格式 + 后半 AOF 格式。

问：RDB 和 AOF 怎么选？持久化时 fork 有什么问题？

答：

场景	推荐
可容忍分钟级丢失	RDB (定时快照)
数据安全优先	AOF everysec
兼顾	混合持久化

fork 问题： - 数据量大时 fork 阻塞主线程 (ms~s) - 写密集时 copy-on-write 复制页多，内存可能翻倍
- 优化：关闭 THP、`maxmemory` 控制实例大小

问：AOF 重写是怎么工作的？

答：1. 主进程 fork 子进程（copy-on-write） 2. 子进程根据内存数据生成新 AOF（合并冗余命令） 3. 重写期间新命令写入 AOF 重写缓冲区 4. 完成后追加缓冲区内容，替换旧 AOF

4. 缓存问题（必考）

问：缓存穿透、击穿、雪崩的区别和解决方案？

答：

问题	原因	解决方案
穿透	查不存在的数据，请求打到 DB	布隆过滤器、缓存空值（短 TTL）
击穿	热点 key 过期，大量并发打 DB	互斥锁（只有一个线程查 DB）、逻辑过期
雪崩	大量 key 同时过期或 Redis 宕机	过期时间加随机值、多级缓存、熔断降级、Redis 集群

5. 集群

问：Redis 主从复制原理？

答：1. 从库发送 PSYNC 命令 2. 首次：全量同步（主库 bgsave RDB → 发送 → 从库加载） 3. 断线重连：增量同步（复制积压缓冲区 repl_backlog + offset）

问：哨兵（Sentinel）的作用？

答：1. 监控：检测主从节点是否正常 2. 通知：故障时通知客户端 3. 自动故障转移：选举 Leader 哨兵 → 从库中选新 Master → 通知客户端

选主算法：类似 Raft，先选 Leader 哨兵 → 从库中选新 Master（优先级 > offset > runid）。

问：Redis 主从和 Cluster 能保证强一致吗？

答：不能强一致，默认 最终一致。

模式	一致性
主从	异步复制，主库写入后从库可能延迟
哨兵	故障转移期间可能丢最后几条写
Cluster	异步复制 + 槽迁移窗口

需要更强一致：Wait 命令（等待 N 个从库确认）、Redis 企业版或业务层补偿。

问：Cluster 客户端怎么知道访问哪个分片？

答：1. 客户端缓存 slot → 节点 映射表 2. $\text{CRC16}(\text{key}) \& 16383$ 计算 slot 3. 访问错误节点时返回 MOVED（永久迁移）或 ASK（临时迁移），客户端更新映射

Smart Client（如 Lettuce、JedisCluster）自动处理重定向。

问：Redis Cluster 分片原理？

答：- 16384 个哈希槽（slot） - $\text{slot} = \text{CRC16}(\text{key}) \& 16383$ - 每个节点负责一部分槽 - 客户端访问错误节点时返回 MOVED/ASK 重定向

6. 分布式锁（重点）

问：Redis 分布式锁的实现原理？加锁要满足哪三个条件？

答：Redis 是共享存储、读写性能高，适合实现分布式锁。

加锁三个条件：1. 互斥：同一时刻只有一个客户端持有锁 → `SET key value NX` 2. 防死锁：客户端崩溃时锁能自动释放 → `EX/PX` 设置过期时间 3. 防误删：锁 value 标识持有者，解锁时校验 → `UUID + Lua 脚本`

加锁：

```
SET lock:order:1001 "uuid-xxx" NX EX 30
# NX: key 不存在才设置; EX 30: 30 秒过期
```

解锁（必须 Lua，保证「判断 + 删除」原子性）：

```
if redis.call("get", KEYS[1]) == ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0
end
```

为什么不用 GET + DEL 两步：两步之间锁可能过期被他人获取，你会删掉别人的锁。

问：Redis 分布式锁有哪些常见问题？怎么解决？

答：

问题	原因	解决方案
误删锁	线程 A 超时，线程 B 加锁，A finally 删了 B 的锁	value 用 UUID，Lua 校验后再删
锁过期业务未完成	EX 30s 但业务跑 60s	Redisson 看门狗续期；或合理评估 TTL
不可重入	同一线程重复加锁会失败	Redisson 可重入锁（Hash 记录重入次数）
主从切换丢锁	主库加锁成功但未同步到从库，主宕机，从升主后锁丢失	RedLock；或 ZK/Etcd
非原子加锁	SETNX + EXPIRE 两条命令	必须一条 SET NX EX

问：Redisson 看门狗（Watchdog）是什么？

答：Redisson 加锁时 默认不指定 leaseTime，Watchdog 后台线程每 10s (leaseTime/3) 续期一次，默认 leaseTime 30s，业务未完成则自动续期。

指定了 leaseTime 则不会启动 Watchdog，到期自动释放。

```
RLock lock = redisson.getLock("lock:order:1001");
lock.lock();           // 看门狗续期
// lock.lock(10, TimeUnit.SECONDS); // 10s 后释放，无看门狗
try {
    // 业务逻辑
} finally {
    lock.unlock();
}
```

原理：Lua 脚本加锁，Hash 结构 key → {线程ID：重入次数}，支持可重入。

问：RedLock 算法是什么？有什么争议？

答：RedLock（Redis 作者 Antirez）：在 N 个独立 Redis Master（通常 5 个）上依次加锁，过半成功且总耗时 < 锁有效期，才算加锁成功。

步骤：1. 记录当前时间 T1 2. 依次向 N 个节点 SET key value NX PX ttl 3. 成功数 ≥ N/2+1 且 T2-T1 < 有效时间 → 成功 4. 失败则向所有节点发起解锁

Martin Kleppmann 质疑：– 依赖系统时钟，时钟跳变可能导致锁同时失效 – GC 长时间 STW 后线程恢复，可能误以为锁仍有效 – 主从异步复制下，RedLock 仍可能丢锁

生产建议：

方案	适用
单 Redis + Redisson 看门狗	大多数业务，可接受极低概率问题
RedLock	多独立 Master，要求较高
ZooKeeper / Etcd	强一致、临时顺序节点，金融级

问：Redis 锁 vs ZooKeeper 锁怎么选？

答：

对比	Redis	ZooKeeper / Etcd
性能	高（内存）	较低（网络+磁盘）
一致性	最终一致，主从可能丢锁	强一致（ZAB/Raft）
实现	SET NX + Lua	临时顺序节点，最小序号获锁
续期	看门狗 / TTL	会话心跳
场景	高并发、允许极低失败率	强一致、分布式协调

问：什么场景需要分布式锁？

答：需要：多进程/多机器对 同一资源 互斥访问 – 库存扣减、订单号生成 – 定时任务集群只跑一个实例 – 缓存击穿重建（互斥查 DB） – 支付防重复提交

不需要：– 单 JVM 内 → `synchronized` / `ReentrantLock` – 数据库能扛 → `SELECT FOR UPDATE`（性能差） – 可接受最终一致 → 消息队列串行消费

秒杀优化：纯 Redis 锁串行化吞吐低，可用 分段锁（`lock:stock:{skuId % 10}`）或 Redis 原子扣减 + MQ 异步落库。

问：用 Redis 锁实现缓存击穿互斥重建？

答：热点 key 过期时，大量请求打 DB。用锁保证 只有一个线程 查 DB 重建缓存：

```
String lockKey = "lock:cache:" + key;
String val = redis.get(key);
if (val == null) {
    if (redis.set(lockKey, "1", "NX", "EX", 10)) {
        try {
            val = db.query(key);
            redis.setex(key, 3600, val);
        } finally {
            redis.eval(unlockScript, lockKey, uniqueId);
        }
    } else {
        Thread.sleep(50); // 等待其他线程重建
        val = redis.get(key);
    }
}
```

与 逻辑过期 对比：逻辑过期不阻塞，返回旧值 + 异步更新；互斥锁会短暂等待。

问：Redis 除了缓存还有哪些应用？

答：

场景	用法
缓存	Cache Aside, 加速读
分布式锁	SET NX EX + Lua
计数器	INCR 限流、点赞数
排行榜	ZSet
消息队列	List LPUSH/BRPOP、Stream
会话	Hash 存 Session
地理位置	GEO 命令
布隆过滤器	防穿透 (RedisBloom)
延迟队列	ZSet score=时间戳

7. 事务

问：Redis 事务 (MULTI/EXEC) 了解吗？

答：– MULTI 开启，EXEC 执行，DISCARD 取消 – 命令入队，不执行；EXEC 时一次性执行 – 不保证原子性：某条命令失败不影响其他命令 – 不支持回滚 – 用 WATCH 实现乐观锁 (CAS)

问：除了 Lua，还有什么方式保证 Redis 原子性？

答：

方式	说明
单条命令	GET/SET/INCR/DECR 等本身原子 (单线程)
Lua 脚本	多条命令整体执行，推荐
MULTI/EXEC	批量执行，某条失败不回滚，原子性弱于 Lua
Redis 模块	自定义原子操作

分布式锁解锁、库存扣减等必须用 Lua 保证「判断 + 修改」原子性。

8. 过期删除与淘汰

问：Redis 过期 key 怎么删除？

答：1. 惰性删除：访问时检查是否过期 2. 定期删除：每秒随机抽查部分 key

过期删除 vs 内存淘汰：

对比	过期删除	内存淘汰
触发	key 设置了 TTL	内存达到 maxmemory
目的	清理过期 key	腾出空间给新写入
策略	惰性 + 定期	LRU/LFU/Random 等

注意：过期 key **不会立即删除**，惰性访问时才删或定期抽查，可能短暂占用内存。

问：内存满了怎么办？淘汰策略？

答： maxmemory-policy： - noeviction：不淘汰，写操作报错（默认） - allkeys-lru：所有 key 中 LRU 淘汰 - volatile-lru：设了过期时间的 key 中 LRU 淘汰 - allkeys-random / volatile-random - volatile-ttl：淘汰 TTL 最短的 - allkeys-lfu / volatile-lfu（Redis 4.0+）

9. 跳表 vs B+ 树

问：Redis 跳表和 MySQL B+ 树有什么区别？为什么不同场景不同选择？

答：

对比	Redis 跳表	MySQL B+ 树
存储	纯内存	磁盘 + 内存（Buffer Pool）
节点连接	多层指针 + 链表	页内指针 + 叶子链表
扇出	较小（~32）	很大（一页 16KB，数百 key）
范围查询	底层链表遍历	叶子链表遍历
实现复杂度	简单	复杂（页分裂/合并）
并发	单线程无锁	需 latch/B+ 树锁

为什么 Redis 用跳表： - 内存操作，树高不是瓶颈 - 实现简单，范围查询 $O(\log N) + O(k)$ - ZSet 还需支持 rank（排名），跳表按 span 字段可 $O(\log N)$ 求排名

为什么 MySQL 用 B+ 树： - 磁盘 IO 是瓶颈，需要 **高扇出** 降低树高 - 非叶子节点不存数据，一页存更多 key，减少 IO 次数

10. 压缩列表与连锁更新

问：什么是压缩列表（ziplist）？有什么问题？

答： 紧凑连续内存，存储小 Hash/List/ZSet： - 每个 entry 存 prevlen + encoding + data - 节省内存，适合元素少、值小的场景

阈值（Redis 7.x 前）： - Hash：元素 ≤ 512 且值 ≤ 64 字节 - ZSet：元素 ≤ 128 且值 ≤ 64 字节

升级：Redis 7.0 用 listpack 替代 ziplist，解决连锁更新。

问：什么是连锁更新（Cascade Update）？

答：ziplist 中每个 entry 的 **prevlen** 记录前一个 entry 长度： – 前 entry < 254 字节：prevlen 占 1 字节 – 前 entry ≥ 254 字节：prevlen 占 5 字节

连锁：插入/删除导致某 entry 长度变化 → 后续 entry 的 prevlen 需扩展/收缩 → 可能引发后续连锁。

最坏： $O(n^2)$ 复杂度，如大量连续 250~253 字节的 entry，一个插入触发全部更新。

解决：listpack 改为 **backlen** 变长编码，只记录自身长度，不依赖前 entry。

11. 渐进式 rehash

问：Redis 哈希表扩容的渐进式 rehash 过程？

答：Redis 字典（dict）有两个哈希表 ht[0]、ht[1]：

触发扩容：负载因子 ≥ 1（used/size）或 正在进行 BGSAVE 且 ≥ 5。

过程：1. 为 ht[1] 分配 2 倍大小空间 2. 设置 **rehashidx = 0**，标记 rehash 开始 3. **渐进 rehash**：每次 CRUD 命令时，顺带迁移 ht[0] 中 rehashidx 指向的桶到 ht[1] 4. 同时 **定时任务** 也帮忙迁移 5. ht[0] 全部迁移完 → ht[1] 变为 ht[0]，ht[1] 置 NULL

优点：避免一次性 rehash 阻塞 Redis（ $O(n)$ 停顿）。

缩容：负载因子 < 0.1 时缩到最小适用 size。

12. 大 Key 与热 Key

问：什么是大 Key 和热 Key？有什么危害？

答：大 Key： – String > 10KB（或业务定义阈值） – Hash/List/Set/ZSet 元素 > 5000

危害：阻塞单线程（删除/序列化慢）、内存不均、主从同步延迟、集群迁移卡顿。

热 Key： – 某 key QPS 极高（如明星微博、秒杀商品）

危害：单节点 CPU 飙高、带宽打满、缓存击穿。

问：大 Key 和热 Key 如何解决？

答：大 Key：

方案	说明
拆分	大 Hash 拆为 key:1 、 key:2
压缩	存 gzip 压缩后的值
异步删除	Redis 4.0+ UNLINK 非阻塞删除
定期扫描	redis-cli --bigkeys 发现

热 Key:

方案	说明
本地缓存	JVM 一级缓存 (Caffeine)
多副本	热 key 加随机后缀 key:{1~N} , 分散到不同 slot
读写分离	从库分担读
监控	Redis 4.0+ redis-cli --hotkeys (需 LFU 策略)

13. 缓存一致性

问: Cache Aside 模式是什么? 有哪些一致性问题?

答: 读: 先读缓存 → miss 则读 DB → 写缓存 写: 先更新 DB → 删除 缓存 (而非更新)

为什么删缓存而非更新: 避免并发写导致缓存与 DB 不一致; 删缓存 lazy load 更简单。

不一致窗口: 删缓存前, 其他线程可能读到旧缓存。解决: 1. 延迟双删: 删 → 更新 DB → sleep → 再删 2. 消息队列异步删缓存 3. Canal 监听 binlog

问: Canal + MQ 如何保证缓存一致性?

答:

MySQL 更新 → binlog → Canal 监听 → 发 MQ → 消费者删 Redis 缓存

流程: 1. 业务更新 DB, 不直接操作缓存 2. Canal 伪装成 MySQL slave, 接收 binlog 3. 解析 binlog, 获取变更的表和主键 4. 发送 MQ 消息 5. 消费者根据 key 规则 删除 Redis 对应缓存

优点: 解耦、最终一致、DB 为准。 注意: MQ 消费失败需重试; 删缓存幂等。

14. 布隆过滤器

问：布隆过滤器原理和应用场景？

答：原理：bit 数组 + 多个 hash 函数。添加元素时，各 hash 位置置 1；查询时全为 1 则 **可能存在**，有 0 则 **一定不存在**。

特点：– 有误差率（不存在误判为存在），无漏判 – 空间效率极高 – 不支持删除（Counting Bloom Filter 可以）

Redis 实现：BF.ADD、BF.EXISTS（RedisBloom 模块）。

场景：– 防缓存穿透：查 Redis 前先过布隆，不存在直接返回 – 爬虫 URL 去重 – 垃圾邮件过滤

参数：预期元素数 n 、误差率 p ，可计算最优 bit 数组大小 m 和 hash 函数个数 k 。

15. 秒杀设计

问：如何用 Redis 设计秒杀系统？

答：核心挑战：高并发读、瞬间大写、超卖、DB 压力。

架构：

1. 前端：按钮置灰、验证码/答题防刷
2. 接入层：Nginx 限流、网关熔断
3. Redis 层：
 - 预热：秒杀开始前将库存写入 Redis（DECR 原子扣减）
 - 校验：Lua 脚本原子「判断库存 → 扣减 → 记录用户」
 - 去重：Set 记录已购买用户，防止重复
4. 异步：扣减成功后发 MQ，消费者异步创建订单、写 DB
5. 兜底：DB 最终库存校验，超卖回滚

Lua 脚本示例：

```
if tonumber(redis.call('get', KEYS[1])) > 0 then
    if redis.call('sismember', KEYS[2], ARGV[1]) == 0 then
        redis.call('decr', KEYS[1])
        redis.call('sadd', KEYS[2], ARGV[1])
        return 1
    end
end
return 0
```

其他：– 分段库存：stock:{0~9} 分散热点 – 逻辑过期：商品数据不过期，异步更新 – 熔断：Redis/DB 异常时快速失败

16. 其他深入

问：Redis 单线程为什么还这么快？6.0 多线程影响什么？

答：瓶颈不在 CPU 计算，而在 内存访问 和 网络 IO。单线程避免锁竞争和上下文切换。

6.0 多线程只并行 网络读写 (read/write query buffer)，命令执行仍单线程，保证原子性不变。

问：Redis 持久化时 fork 会有什么问题？

答：fork 使用 copy-on-write：– 子进程做 RDB/AOF 重写 – 主进程继续处理写请求，修改的页才复制

问题：– 数据量大时 fork 耗时（可能阻塞 ms~s 级）– 写密集时 COW 复制页多，内存翻倍风险

优化：`echo never > /sys/kernel/mm/transparent_hugepage/enabled` 避免大页复制；控制单实例内存 $\leq 10\text{GB}$ 。

图解加深

1. 跳表结构

理解要点：多层索引 + 底层有序链表，查找从顶层开始「跳跃」，ZSet 的 score 排序靠此实现。

2. 渐进式 rehash

两个哈希表并存，rehashidx 逐步推进，读写同时服务，避免一次性迁移卡顿。

3. 缓存一致性

推荐 Cache Aside + Canal 删缓存，DB 为准，最终一致。

大厂追问

1. Redis 和 Memcached 区别？

Redis 丰富数据结构、持久化、主从、集群；Memcached 纯 KV、多线程、内存利用率略高。Redis 功能全面，业界主流。

2. 分布式锁 Redlock 争议？

Redis 作者 Antirez 提出 Redlock（多 Master 过半成功）。Martin Kleppmann 质疑时钟漂移和 GC 暂停可能导致不安全。生产环境 Redisson 单节点+看门狗 或 ZooKeeper/Etcd 更可靠。

10. 加锁后业务执行超过 TTL 怎么办？

Redisson Watchdog 自动续期；或手动评估业务 P99 设 TTL + 业务内校验；绝不用无限期锁。

11. SETNX + EXPIRE 两步为什么不行？

两步非原子，SETNX 成功后进程崩溃，锁永不过期 → 死锁。必须 `SET key value NX EX`。

12. 主从架构下 Redis 锁安全吗？

主库加锁后宕机，从库未同步则锁丢失，新主无锁记录 → 双写。高要求场景用 RedLock 或 ZK。

3. 为什么 Cluster 是 16384 个槽？

CRC16 结果 16 bit，取模 16384。节点间心跳包携带槽信息，16384 位约 2KB，太大则心跳负担重。16384 对千级节点足够。

4. Redis 内存淘汰 LRU 和 LFU 区别？

LRU：淘汰最久未访问。LFU：淘汰访问频率最低。LFU 更适合热点明显的场景（如秒杀），Redis 4.0+ 推荐 LFU。

5. 如何保证 Redis 和 DB 双写一致？

强一致很难。推荐：**先更新 DB，再删缓存** + Canal 异步删 + 重试；或读写都走 DB，缓存只做加速，容忍短暂不一致。

6. 渐进式 rehash 期间读请求怎么处理？

同时查 ht[0] 和 ht[1]：若 key 在 ht[0] 且 rehashidx 未迁移，从 ht[0] 读；已迁移则从 ht[1] 读。写操作顺带迁移桶。

7. AOF 重写期间新命令会丢吗？

不会。重写期间新写命令进 **AOF 重写缓冲区**，子进程完成后追加到 new AOF 再替换。

8. Redis 单节点 + MySQL 单节点大概多少 QPS？

视业务而定：Redis 读 10 万+ QPS；MySQL 简单查询数千 QPS。瓶颈通常在 DB，Redis 做缓存可大幅提升整体吞吐。

9. 缓存击穿「逻辑过期」方案是什么？

热点 key 不设 TTL，value 内嵌逻辑过期时间；发现过期时 **互斥锁** 只让一个线程查 DB 重建，其他线程返回旧值，避免同时打 DB。